

A Industrial Project Report on
STOCK PRICE PREDICTION USING MACHINE LEARNING

Submitted to

Jawaharlal Nehru Technological University, Hyderabad

Submitted in partial fulfillment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

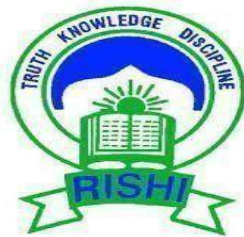
By

VANI SAI DEEPIKA	226P1A05H2
VAISHNAVI SUROJU	226P1A05H1
PUNNANAVYA	226P1A05C7
VARSHA RATHNAM	226P1A05H3

Under the guidance of

Dr. Archana Patil, B.Tech,M.Tech, Ph.D.

Associate Professor, & HOD Department of
CSE



Department of Computer Science and Engineering

Rishi MS Institute of Engineering and Technology for Women

Approved by AICTE, Affiliated to JNTUH

Nizampet Cross Road, near JNTUH, Kukatpally, Hyderabad, Telangana 500085

2024-2025



Rishi M.S. Institute of Engineering & Technology for Women

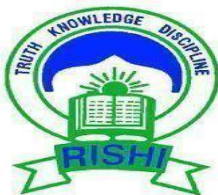
(Approved by A.I.C.T.E., & Affiliated to J.N.T.U.H.)

(In Memory of "BHARAT RATNA" Mrs. M.S. Subbulakshmi)

Near J.N.T.U.H Metro Station, Nizampet 'X' Road, Kukatpally, Hyderabad - 500 085.

E-mail: rishims2009@gmail.com, Phone: 040-23892878, Fax: 040-23892858.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project entitled “**STOCK PRICE PREDICTION USING MACHINE LEARNING**” being submitted by

VANI SAI DEEPIKA **226P1A05H2**

VAISHNAVI SUROJU **226P1A05H1**

PUNNANAVYA **226P1A05C7**

VARSHA RATHNAM **226P1A05H3**

In partial fulfillment of the award of degree of BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING to Rishi MS Institute of Engineering & Technology for women, affiliated to the Jawaharlal Nehru Technological University, Hyderabad during the year 2024-2025. is a record of bona fide piece of work, undertaken under the supervision of the undersigned.

Supervisor's Signature

Dr. Archana Patil, B.Tech,M.Tech,Ph.D.

Associate Professor,& HOD

Department of CSE

Head of the Department

Dr. ArchanaPatil,B.Tech,M.Tech,Ph.D.

Associate Professor,&

HOD Department of CSE

Submitted for Viva Voice Examination held on _____

External Examiner

ACKNOWLEDGEMENT

This report will certainly not be completed without due acknowledgements paid to all those who have helped us in doing our Major Project work.

We would like to express our sincere thanks to our Secretary **Ms. Rajasree**, for her constant supervision and encouragement, which helped us in completing this work successfully.

We are expressing our sincere gratitude to our Principal **Dr. Archana Patil** for her timely suggestions, which helped us to complete this work successfully.

It's our privilege to thank **Dr. Archana Patil**, Head of the Department for her encouragement during the progress of this work.

We derive great pleasure in expressing our sincere gratitude to our Project Co-Ordinator **T. Sowjanya & M. Tejaswini** for their encouragement and timely suggestions during the progress of this work.

We express our sincere thanks to our Guide **Dr. Archana Patil**, for giving us moral support, kind attention and valuable guidance to us throughout this work.

We are thankful to our **Rishi MS Institute of Engineering and Technology for Women** for providing the required facilities during the Major Project work.

We would like to thank our parents and our friends for being supportive all the time, and we are very much obliged to them.

VANI SAI DEEPIKA	226P1A05H2
VAISHNAVI SUROJU	226P1A05H1
PUNNANAVYA	226P1A05C7
VARSHA RATHNAM	226P1A05H3



Rishi M.S. Institute of Engineering & Technology for Women

(Approved by A.I.C.T.E., & Affiliated to J.N.T.U.H.)

(In Memory of "BHARAT RATNA" Mrs. M.S. Subbulakshmi)

Near J.N.T.U.H Metro Station, Nizampet 'X' Road, Kukatpally, Hyderabad - 500 085.

E-mail: rishims2009@gmail.com, Phone: 040-23892878, Fax: 040-23892858.

Department of Computer Science and Engineering

Vision & Mission of the Department

Vision of the Department

To promote center of excellence in sustainable academic environment to impart technical skills and strong research platform to prepare global leaders

Mission of the Department

1. To create good academic platform with modern teaching and learning methodologies.
2. To enhance leadership qualities with ethics, values and sense of teamwork for professional excellence.
3. To inculcate research culture by encouraging projects in advanced technologies through industry interactions.

PROGRAM OUTCOMES (POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10.Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11.Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12.Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSO'S)

1. Model computational problems by applying mathematical concepts and design solutions using suitable data structures and algorithmic techniques.
2. Design and develop solutions by following standard software engineering principles and implement by using suitable programming languages and platforms
3. Develop system solutions involving both hardware and software modules

**STOCK
PRICE PREDICTION
USING MACHINE
LEARNING**

S.NO	CONTENTS	PGNO
i	Abstract	i
ii	List of Figures	ii
iii	List of Acronyms	iii
1	INTRODUCTION 1.1 Problem Statement 1.2 Scope of research 1.3 Existing System 1.3.1 Disadvantages of existing system 1.4 Proposed System 1.4.1 Advantages of proposed system	1-5
2	LITERATURE REVIEW	6-7
3	METHODOLOGIES 3.1 System Architecture 3.2 Algorithms 3.2.1 LSTM (Deep Learning Model)	8-12
4	SOFTWARE REQUIREMENTS SPECIFICATIONS 4.1 Functional Requirements 4.2 Performance Requirements 4.3 Software Requirements 4.4 Hardware Requirements 4.5 Languages / Technologies Used	13-17
5	SYSTEM DESIGN 5.1 Introduction To UML	18-23

	5.2 UML diagrams 5.2.1 Use-Case Diagram 5.2.2 Class Diagram 5.2.3 Activity Diagram 5.2.4 Sequence Diagram 5.2.5 Data Flow Diagram	
6	IMPLEMENTATION 6.1 Code	19-26
7	RESULTS 7.1 Output	27-33
8	TESTING 8.1 System Testing 8.2 Types of Testing 8.2.1 Unit Testing 8.2.2 Integration Testing 8.2.3 Black Box Testing 8.2.4 White Box Testing 8.2.5 Performance Testing 8.2.6 Acceptance Testing 8.3 Test Cases	34-42
9	CONCLUSION	43
10	FUTURE ENHANCEMENT	44-45
11	REFERENCES	46

ABSTRACT

This project involves the development of a stock price prediction web application based on deep learning and time-series forecasting methods to forecast future stock prices. The application is implemented with a modular, scalable design and coded using Python, with Streamlit as the frontend framework to allow an interactive and user-friendly interface. The main goal is to allow users, whether they are investors, students, or researchers, to enter a stock ticker and a range of dates and get a visual projection of future stock prices based on historical data and technical analysis. When a user interacts with the app, it starts a sequence of backend operations that start with the grabbing of historical stock data via the Yahoo Finance API. This information is subsequently cleaned up and augmented with the addition of critical technical indicators, including the Relative Strength Index (RSI) and Moving Average Convergence Divergence (MACD), which are found in widespread financial analysis to determine momentum and trend direction. The augmented dataset is normalized through various techniques of normalization to facilitate effective learning by the LSTM model from the time-series data. The prediction engine utilizes a Long Short-Term Memory (LSTM) neural network, a recurrent neural network optimized for sequential data such as stock prices. The model has been pre-trained on historical market data and can identify intricate temporal patterns and relationships. When new input data is given to the application, the LSTM model processes the data and gives a sequence of future price predictions.

Then, these results are dynamically visualized using understandable plots, enabling users to compare past data to forecasted trends within the Streamlit dashboard. This not only makes the forecasting process easily accessible but also informative for non-technical users. The system is built with a modular folder organization, and each element like data loading, preprocessing, calculating technical indicators, modeling, and visualization has its own Python module. The modular approach encourages reusability of code, simplicity in maintenance, and extensibility in the future. In short, this stock price forecasting app combines machine learning, financial metrics, and interactive web technology to provide an all-encompassing tool for predicting stock market trends. It illustrates the way deep learning can be used in practical financial issues and as a base system that could be extended further with functionalities such as multi-stock comparison, model training interfaces, or macroeconomic sentiment data integration.

LIST OF FIGURES

FIG.NO	FIG NAME	PG.NO
3.1.1	System Architecture	10
5.2.1	Use case diagram	25
5.2.2	Class diagram	25
5.2.3	Activity diagram	26
5.2.4	Sequence diagram	27
5.2.5	Data Flow diagram	28
7.2.1	Main Page	35
7.2.2	Manual Inputs (Stock Id, Start Date, End Date, Days to be predicted)	36
7.2.3	Technical Indicators used. (RSI, MACD)	36
7.2.4	Data Preview for Apple Company stock	37
7.2.5	Model prediction on existing data	37
7.2.6	The predicted cost of the stock	37
7.2.7	Download the predicted data	38
7.2.8	Stock details for TCS company	38
7.2.9	Data preview of TCS stock	38
7.2.10	Technical Indicators for TCS	39
7.2.11	TCS stock prediction on existing data	39
7.2.12	30-day cost prediction for TCS stock	40

LIST OF ACRONYMS

SN NO	Acronyms	Description
1	LSTM	Long Short-Term Memory
2	RSI	Relative Strength Index
3	MACD	Moving Average Convergence Divergence
4	CSV	Comma-Separated Values
5	API	Application Programming Interface
6	UI	User Interface
7	ML	Machine Learning
8	GPU	Graphics Processing Unit
9	IDE	Integrated Development Environment
10	BILSTM	Bidirectional Long -Short term Memory

CHAPTER 1

1. INTRODUCTION

1.1 Problem Statement

The stock market is inherently volatile, driven by numerous unpredictable factors including economic indicators, company performance, market sentiment, and geopolitical events. For investors and financial analysts, accurately predicting stock prices remains a complex and high-stakes challenge. Traditional methods of analysis, such as fundamental and technical analysis, require significant manual effort, expertise, and are often limited in their ability to model nonlinear and time-dependent patterns within stock price movements.

Moreover, there is a lack of accessible, real-time tools that utilize advanced machine learning techniques to forecast future stock prices while providing user-friendly interfaces suitable for non-technical users. Existing financial prediction platforms are often either too simplistic — lacking depth in analysis — or too complex for casual investors or learners to use effectively.

This project aims to bridge that gap by developing a modular, interactive web application that employs a Long Short-Term Memory (LSTM) neural network to predict stock prices based on historical data and technical indicators. By automating data retrieval, preprocessing, feature engineering, and prediction, the system offers a streamlined and accurate approach to short-term forecasting. The goal is to provide a reliable and insightful tool for users to visualize future stock trends, reduce reliance on manual charting, and make more informed investment decisions.

1.2 Scope of Research

The scope of this research encompasses the design, development, and evaluation of a machine learning-based system for predicting future stock prices using historical market data and technical indicators. The application focuses on short-term forecasting and is developed as a web-based tool to enhance accessibility and usability for a broad range of users, including retail investors, data science students, and financial analysts.

Key areas covered within the scope include:

- **Data Collection and Integration:** Real-time stock market data is sourced from the Yahoo Finance API, focusing on attributes such as closing prices, volume, and other relevant market indicators. The system supports dynamic input for multiple stock tickers and customizable date ranges.

- **Feature Engineering:** Technical indicators such as the Relative Strength Index (RSI) and Moving Average Convergence Divergence (MACD) are calculated to enrich the dataset. These indicators assist in capturing momentum and trend-related signals in price movements.
- **Time-Series Forecasting with LSTM:** The core of the prediction model is a Long Short-Term Memory (LSTM) neural network, specifically chosen for its ability to learn temporal dependencies and sequential patterns in historical stock data. The research explores model architecture, training parameters, and performance metrics.
- **Modular System Architecture:** The backend is organized into modular components — including data loading, preprocessing, modeling, and visualization — to ensure scalability, maintainability, and ease of enhancement in future iterations.
- **User Interface and Visualization:** A responsive and interactive user interface is developed using Streamlit, allowing users to input queries and view predictions with clear visual representations. This makes the system approachable even for users without technical or financial backgrounds.
- **Evaluation and Output Analysis:** The accuracy of predictions is evaluated using appropriate metrics (e.g., RMSE, MAE), and the results are visually compared with actual historical data to assess the model's effectiveness.

1.3 Existing System

Currently, several systems and platforms exist for analyzing and predicting stock prices. These include both traditional financial analysis tools and modern machine learning-based models. However, each has its own set of limitations that restrict its accessibility, accuracy, or usability for broader audiences.

1. Traditional Financial Analysis Tools

- Rely on fundamental analysis (company earnings, P/E ratios) and technical analysis (charts, moving averages).
- Tools like Yahoo Finance, TradingView, and MetaTrader provide charting features, indicators, and historical data.
- Require manual effort and expertise to interpret trends and make predictions.
- Do not leverage machine learning for forecasting, limiting their adaptability to complex,

nonlinear market behavior.

2. Automated Forecasting Models

- Some platforms offer AI-based predictions using statistical models or pre-built ML tools.
- Tools such as QuantConnect, Alpaca, or proprietary trading bots use complex algorithms behind paywalls or APIs.
- These are often black-box systems, providing little transparency into how predictions are made.
- Many require programming knowledge or are locked behind expensive subscriptions, making them inaccessible to casual users or students.

3. Academic Implementations

- Several research papers and GitHub repositories demonstrate the use of LSTM, ARIMA, or Prophet models for stock prediction.
- However, these projects are typically not user-friendly, lack a frontend interface, and are not easily deployable.
- They serve more as proof-of-concept models rather than practical tools for day-to-day use.

1.3.1 Disadvantages of the Existing System

- **Manual Analysis Required:** Traditional platforms rely heavily on user-driven technical analysis, demanding prior financial knowledge and experience.
- **Limited Use of AI/ML:** Many tools do not utilize advanced machine learning algorithms like LSTM, failing to capture complex patterns in time-series data.
- **Inaccessible for Beginners:** Existing models often require programming skills or financial expertise, creating a barrier for students, casual investors, or self-learners.
- **Lack of Real-Time Forecasting:** Most academic or open-source models are not deployed as real-time apps — they function more as offline experiments.

- **Black Box Predictions:** AI-powered commercial tools provide predictions without transparency on how they were derived, reducing trust and interpretability.
- **Poor User Experience:** Few platforms offer an integrated, clean, and interactive interface that bridges the gap between raw data and understandable insights.
- **No Modular or Extensible Design:** Existing open-source models are often written in monolithic scripts, making them hard to maintain, customize, or scale.
- **No Integration of Indicators + Deep Learning:** Most systems either use technical indicators or deep learning — not both. This leads to less informed or less robust forecasting.

1.4 Proposed System

The proposed system is an interactive, modular, and intelligent web application that predicts future stock prices using a Long Short-Term Memory (LSTM) neural network, enriched with technical indicators. It aims to bridge the gap between traditional financial tools and modern AI-powered forecasting by offering a user-friendly yet powerful platform.

Key Features of the Proposed System

- **LSTM-Based Price Prediction**

Utilizes a deep learning model (LSTM) to learn from historical price trends and forecast future stock prices with improved accuracy.

- **Technical Indicator Integration:**

Enhances prediction accuracy by incorporating popular technical indicators such as:

- Relative Strength Index (RSI)
- Moving Average Convergence Divergence (MACD)

- **Modular Code Architecture:**

The app is designed in a modular format with clear separation of concerns:

- `data_loader.py` – Data fetching and preparation
- `indicators.py` – Calculation of RSI, MACD
- `model.py` – LSTM model loading and inference
- `utils.py` – Helper functions for scaling, splitting, plotting
- `app.py` – Streamlit-powered user interface

- **Real-Time Data Fetching:**

Uses Yahoo Finance API to pull up-to-date stock data dynamically based on user input.

- **Interactive UI with Streamlit:**

Offers a lightweight, responsive frontend where users can:

- Enter stock ticker and date range
- View actual vs. predicted prices on interactive charts

- **Easy to Use & Deploy:**

Designed to be run locally or hosted with minimal setup. No advanced technical skills required from the user.

1.4.1 Advantages of Proposed System

- **User-Friendly:** Ideal for both technical and non-technical users.
- **Accurate:** Uses time-series deep learning (LSTM) enhanced with indicator signals.
- **Customizable:** Modular design allows for easy extension (e.g., adding more indicators, training models).
- **Efficient:** Fast data processing and real-time visualization.
- **Transparent:** Unlike black-box commercial tools, the system is open and interpretable.

CHAPTER 2

2. LITERATURE REVIEW

1. Multi-Agent Stock Prediction Systems:

Machine Learning Models, Simulations, and Real-Time Trading Strategies

- ❖ Authors: Daksh Dave, Gauransh Sawhney, Vikhyat Chauhan
- ❖ Year: 2025
- ❖ Source: arXiv Preprint
- ❖ Link: <https://arxiv.org/abs/2502.15853>

This study presents a comprehensive analysis of stock price prediction by leveraging advanced machine learning (ML) and deep learning (DL) techniques to enhance financial forecasting accuracy. The researchers evaluate various recurrent neural network (RNN) architectures, including Long Short-Term Memory (LSTM) networks, Gated Recurrent Units (GRU), and attention-based models. Their findings indicate that attention-based models outperform others by effectively capturing both short and long-term dependencies in stock market data. This work offers valuable insights into AI-driven financial forecasting, providing practical guidance for developing more accurate and efficient trading systems.

2. Stock Price Prediction Using Machine Learning Techniques:

- ❖ Author(s): A.T. Elangovan, R. Vasanthi
- ❖ Year: 2024
- ❖ Source: International Journal of Progressive Research in Engineering Management and Science (IJPREMS)
- ❖ Link:
https://www.ijprems.com/uploadedfiles/paper/issue_9_september_2024/35981/final/fin_ijprems1727366658.pdf

This recent study presents an in-depth comparison of machine learning algorithms applied to stock price prediction. The researchers focused on supervised learning techniques such as Decision Tree, Random Forest, and Support Vector Machines (SVM), highlighting their individual strengths in capturing nonlinear patterns in volatile markets. The study emphasizes the Long Short-Term Memory (LSTM) network for time-series forecasting, demonstrating its

ability to retain memory of previous inputs—crucial for understanding stock movements. Data preprocessing methods like normalization, feature scaling, and moving averages are discussed in detail, making this paper a complete pipeline reference. The paper concludes that while traditional models are helpful, deep learning methods such as LSTM and GRU show promising results in capturing market complexities and delivering higher accuracy in real-time financial environments.

3. A Comprehensive Survey of Predicting Stock Market Trends Using Machine Learning Approaches.

- ❖ Author(s): Iffat Ara Tumpa, Nandita Roy, Md. Nazrul Islam.

- ❖ Year: 2023.

- ❖ Source: AIP Conference Proceedings.

- ❖ Link: <https://pubs.aip.org/aip/acp/article/3107/1/050025/3287938>

This paper surveys offers a holistic survey of ML algorithms and their application in forecasting market trends. It reviews a wide range of methods—from traditional classifiers like SVM and Logistic Regression to advanced ensemble techniques and deep learning models like CNN and RNN. The authors provide critical insights into model interpretability, data reliability, and financial bias, explaining how news sentiment, technical indicators, and social media data enhance prediction accuracy when used alongside historical pricing data. It also explores hybrid models, combining multiple ML techniques to maximize prediction performance and reduce risk. The paper identifies key challenges like overfitting, noise sensitivity, and lack of generalization across markets. This literature acts as a bridge between classical financial theories and modern-day data-driven AI applications.

CHAPTER 3

3. METHODOLOGIES

The methodology of our stock price prediction system integrates user interaction, backend data processing, and deep learning techniques. The system follows a well-defined flow to ensure accuracy and usability:

- 1. User Interface with Streamlit:** The application interface is developed using Streamlit, allowing users to interact with the system via a simple and intuitive web UI.
- 2. Input Phase:** The user inputs the required stock ID and desired date range for prediction. This input is captured through the UI and sent to the backend for processing.
- 3. Click Event Trigger:** On clicking the 'Enter' button, the data is passed to the backend where further operations are performed.
- 4. Backend Processing:** The system handles the user's input by initiating processes to fetch and prepare stock market data.
- 5.Data Fetching:** Real-time or historical stock data is fetched from a relevant data source (such as Yahoo Finance or an internal database) based on the user's input.
- 6.Normalization and Scaling:** To enhance model performance, the fetched data is normalized and scaled. This step ensures that all data points are within the same range, which is crucial for the LSTM model's accuracy.
- 7.Prediction:** using LSTM The processed data is fed into a pre-trained LSTM (Long Short-Term Memory) model. LSTM is a type of Recurrent Neural Network (RNN) suitable for time-series data like stock prices. It learns the pattern from historical data and predicts future trends.
- 8.Visualization of Results:** Finally, the predicted stock prices are displayed along with visual charts for better interpretation. These charts help users to understand the model's output and trends effectively.

3.1 System Architecture

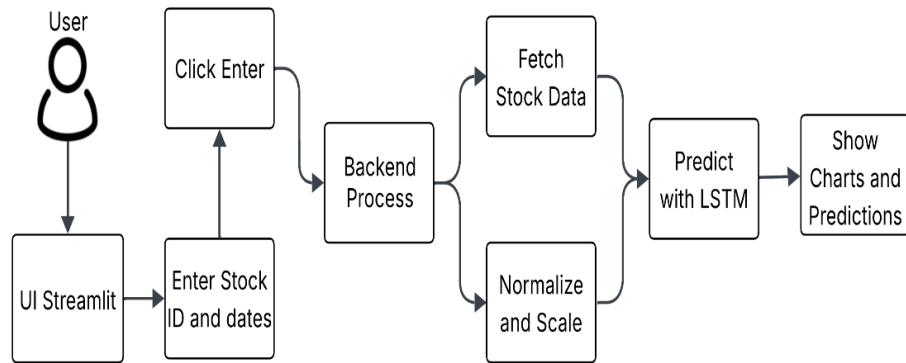


Fig 3.1.1 System Architecture

This architecture outlines the end-to-end workflow of the Stock Price Prediction Web Application, built using Streamlit for UI and LSTM (Long Short-Term Memory) for forecasting future stock prices.

1. User Input Module

The system begins with an interactive frontend where users input:

- The stock ticker symbol (e.g., AAPL, TCS)
- The date range (start and end dates for historical data)
- The number of future days to predict stock prices

This input drives the flow of data throughout the system.

2. Data Loader Module

Using APIs (like yfinance), the application fetches historical stock data such as:

- Open, High, Low, Close prices
- Trading Volume

This raw data forms the foundation for both technical analysis and model training.

3. Technical Indicator Module

The system computes popular technical indicators:

- **RSI (Relative Strength Index)** – detects overbought/oversold conditions

- **MACD (Moving Average Convergence Divergence)** – shows trend strength and momentum

These are not only visualized but can also help in enhancing model accuracy when used as features.

4. Preprocessing Module

Before feeding the data into the ML model, the system performs:

- **Data Scaling:** Uses MinMaxScaler to normalize input features
- **Sequence Generation:** Transforms time-series data into (X, Y) pairs where:
 - X = Past N days of data
 - Y = Price of next day (or next N days)

This formatting is crucial for LSTM to learn temporal patterns effectively.

5. Modeling Module

This is the heart of the architecture:

- **LSTM Model Training:** A sequential deep learning model is trained using the preprocessed data
- **Prediction:** Once trained, the model forecasts the next N days of stock prices

The model leverages the memory power of LSTM to capture long-term dependencies in time-series data.

6. Visualization Module

The system dynamically visualizes:

- Predicted vs. Actual Prices (Line Charts)
- RSI and MACD Trends

These insights help users understand both the quantitative output and market conditions visually.

7. Export/Output Module

The final output can be:

- Viewed in graphs (inside the app)
- Downloaded as CSV for external use or analysis

3.2 Algorithms

In this project, the primary algorithm used for stock price prediction is the Long Short-Term Memory (LSTM) network. The choice of LSTM is motivated by its strength in handling sequential and time-series data, which makes it highly suitable for predicting future stock prices based on historical trends.

1. Long Short-Term Memory (LSTM)

Overview:

LSTM is a type of Recurrent Neural Network (RNN) specifically designed to learn from sequences over time while avoiding common issues like vanishing gradients. It is effective for time-series forecasting due to its ability to maintain long-term dependencies

Functionality:

The LSTM network processes stock data by observing the historical price trends and learning temporal patterns. It uses memory cells, forget gates, input gates, and output gates to decide what to remember and what to forget at each time step.

Advantages in Stock Prediction:

- Captures time dependencies and trend patterns.
- Handles noisy and complex data better than basic RNNs.
- Retains historical context for better forecasting accuracy

Input to LSTM:

Normalized stock price data structured in a sequential format (e.g., sliding window of previous N days).

Output of LSTM:

Predicted stock price for the next time step(s), which is then rescaled to the original value range.

2.Normalization Algorithm (Min-Max Scaling):

Before feeding data into the LSTM, normalization is applied using Min-Max Scaling

3.2.1 LSTM (Deep Learning Model)

The Long Short-Term Memory (LSTM) model is a powerful variant of Recurrent Neural Networks (RNNs), specifically designed to handle sequence prediction problems such as stock price forecasting. LSTM is well-suited for modeling time-series data due to its ability to capture long-term dependencies and patterns over time.

Key Features of LSTM:

Memory Cells: Unlike traditional RNNs, LSTM uses memory cells that can retain information for long durations, which helps in learning from long historical sequences.

Gated Architecture: It consists of three primary gates — Input Gate, Forget Gate, and Output Gate — that control the flow of information and manage memory updates efficiently.

Resistance to Vanishing Gradient: LSTM addresses the vanishing gradient problem that affects traditional RNNs, enabling it to learn from long sequences without losing important information.

Why LSTM for Stock Prediction?

- Stock prices are naturally sequential and depend on historical data.
- LSTM can learn complex temporal relationships and trends.
- It adapts well to noisy and non-linear financial data.

Workflow of LSTM in This Project:

1. Input Preparation: Historical stock prices are formatted into sequential windows.
2. Normalization: Data is scaled using Min-Max normalization to improve model efficiency.
3. Model Training: The LSTM model is trained on normalized sequences to learn temporal patterns.
4. Prediction: Given recent input sequences, the model predicts the next value(s) in the sequence.
5. Inverse Scaling: Predicted values are transformed back to the original scale for visualization.

Advantages of Using LSTM:

- Captures both short-term fluctuations and long-term trends.
- Provides better accuracy compared to traditional ML models for time-series data.
- Robust to missing or noisy data often seen in stock market datasets.

CHAPTER 4

4. SOFTWARE REQUIREMENTS SPECIFICATIONS

4.1 Functional Requirements

The following functional requirements define the specific behavior and operations the system must support to fulfill its purpose:

1. User Input

- The system shall allow the user to enter a valid stock ticker symbol (e.g., AAPL, TSLA).
- The system shall allow the user to select a date range for historical data analysis.

2. Data Acquisition

- The system shall fetch historical stock price data in real-time using the Yahoo Finance API based on user input.
- The system shall store this data temporarily for preprocessing and modeling.

3. Data Preprocessing

- The system shall clean and normalize the fetched data using MinMaxScaler or equivalent techniques.
- The system shall split the data into training and testing sets for accurate model inference.

4. Technical Indicator Calculation

- The system shall compute technical indicators like:
 - **Relative Strength Index (RSI)**
 - **Moving Average Convergence Divergence (MACD)**

5. Price Prediction using LSTM

- The system shall use a pre-trained LSTM model to predict future stock prices based on historical data.
- The system shall output a predicted price sequence for the selected date range.

6. Data Visualization

- The system shall display interactive line plots of:
 - Historical stock prices
 - Predicted prices
 - Technical indicators

7. User Interface

- The system shall provide a clean, responsive Streamlit interface to:

- Enter input
- Display visualizations
- Show model results

4.2 Performance Requirements

These requirements define how efficiently and reliably the system should perform under various conditions:

1. Response Time

- The system shall fetch and preprocess historical stock data within 2–3 seconds of user input.
- The model shall generate stock price predictions and visualizations within 2 seconds after data preparation.

2. Prediction Accuracy

- The LSTM model shall achieve a minimum accuracy threshold as evaluated by:
 - Root Mean Square Error (RMSE)
 - Mean Absolute Error (MAE)
- Acceptable RMSE values should be below 10% of the average stock price for the prediction range.

3. Model Inference Time

- The LSTM model shall perform forward prediction (inference) in under 1 second for a single stock input over 60–100 time steps.

4. Interface Responsiveness

- The Streamlit UI shall remain responsive during:
 - Data fetching
 - Indicator calculations
 - Model prediction
 - Chart rendering

5. Visualization Performance

- The system shall plot and render prediction charts and technical indicators without visible lag.
- It shall support smooth and real-time interactivity on standard modern browsers.

6. System Load Tolerance

- The system shall function smoothly on machines with minimum:
 - 4 GB RAM

- Dual-core processor
- Stable internet connection for API data fetching

7. Scalability (Modular Design)

- The system shall support the addition of new indicators, models, or backend modules without impacting overall performance.

4.3 Software Requirements

These software requirements define the tools, platforms, and libraries necessary for developing, running, and deploying the application.

1. Operating System

- Windows 10 or later / Ubuntu 20.04+ / macOS (any recent version)

2. Programming Language

- Python 3.8 or higher

3. Required Python Libraries

- streamlit – Web app interface
- yfinance – Fetching historical stock data
- numpy, pandas – Data manipulation and preprocessing
- matplotlib, plotly, seaborn – Data visualization
- scikit-learn – Scaling and preprocessing
- tensorflow / keras – Building and running the LSTM model
- ta – Technical analysis indicators (RSI, MACD, etc.)
- joblib or pickle – Model saving and loading

You can install them using:

```
bash
```

```
pip install -r requirements.txt
```

4. Development Environment

- VS Code / PyCharm – Code editing
- Jupyter Notebook (optional) – For prototyping and testing
- Git & Git Hub – Version control

5. Framework

- **Streamlit** – For building and deploying the interactive web interface

6. Testing Tools (Optional but Recommended)

- pytest – For unit testing Python modules
- streamlit-testing – For testing UI components (optional)

7. Deployment Options

- Can be run **locally** with `streamlit run app.py`
- Can be deployed to:
 - **Streamlit Cloud**
 - **Heroku / Render / Railway** (with minor configuration)

4.4 Hardware Requirements

These requirements outline the minimum and recommended hardware specifications for developing and running the application efficiently.

1. Minimum Requirements

Component	Specification
Processor	Dual-Core CPU (2.0 GHz or above)
RAM	4 GB
Storage	2 GB free disk space
Display	1366 x 768 resolution
Internet	Stable internet connection (for data fetching from Yahoo Finance)
GPU	Not required (inference only, no training)

2. Recommended Requirements

Component	Specification
Processor	Quad-Core CPU (Intel i5/i7 or AMD Ryzen 5/7)
RAM	8 GB or higher
Storage	10 GB SSD free space
Display	Full HD (1920 x 1080)
Internet	High-speed internet connection
GPU	Optional (only needed for training or extending the model)

- **Device Compatibility**
- Works on laptops and desktops
- Compatible with Windows, Linux, and macOS
- Not designed for mobile or tablet use

4.5 Languages / Technologies Used

This section outlines the core programming languages, tools, frameworks, and libraries used in the development of the stock price prediction application.

1. Programming Languages

- **Python**

Used for backend logic, data processing, model implementation, and app development.

2. Web Framework

- **Streamlit**

Lightweight, Python-based framework for creating interactive data science web applications with minimal code.

3. Machine Learning / Deep Learning

- **TensorFlow / Keras**

Used to build and run the LSTM (Long Short-Term Memory) neural network model for time series prediction.

- **scikit-learn**

Used for data preprocessing tasks like scaling, splitting, and evaluating model performance.

4. Data Handling and Visualization

- **pandas** – For data manipulation and handling

- **numpy** – For numerical operations

- **matplotlib / seaborn / plotly** – For plotting and visualizing stock trends and prediction results

5. Technical Analysis

- **ta-lib / ta**

Used to calculate technical indicators like RSI (Relative Strength Index) and MACD (Moving Average Convergence Divergence)

6. Data Source

- **yfinance**

Fetches real-time and historical stock market data from Yahoo Finance.

7. Supporting Tools

- **joblib / pickle** – For saving and loading trained models

- **Git** – Version control system for collaborative development

- **VS Code / Jupyter Notebook** – Development and prototyping environment

CHAPTER 5

5. SYSTEM DESIGN

5.1 Introduction to UML

Unified Modeling Language (UML) is a standardized visual language used for specifying, visualizing, constructing, and documenting the artifacts of software systems. It provides a set of graphical notations to create abstract models of system structure and behavior.

UML helps in understanding, designing, and communicating the architecture and design of a system before actual development begins. It is especially useful for object-oriented software design.

Purpose of UML in This Project

In the context of the Stock Price Prediction Web App, UML is used to:

- Visualize the interaction between components such as data sources, prediction models, and the user interface.
- Represent the system's architecture and workflow clearly.
- Document the structure of modules and data flow to support modular development.
- Aid in communication between team members, especially during collaborative work.

Common UML Diagrams Used

For this project, the following UML diagrams are most relevant:

1. **Use Case Diagram** – Shows user interactions with the system.
2. **Class Diagram** – Represents classes, attributes, and relationships in the backend modules.
3. **Sequence Diagram** – Visualizes the flow of operations, such as fetching data, running the model, and displaying results.
4. **Activity Diagram** – Describes the workflow of processes like data preprocessing and prediction.

5.2 UML Diagrams

5.2.1 Use-Case Diagram

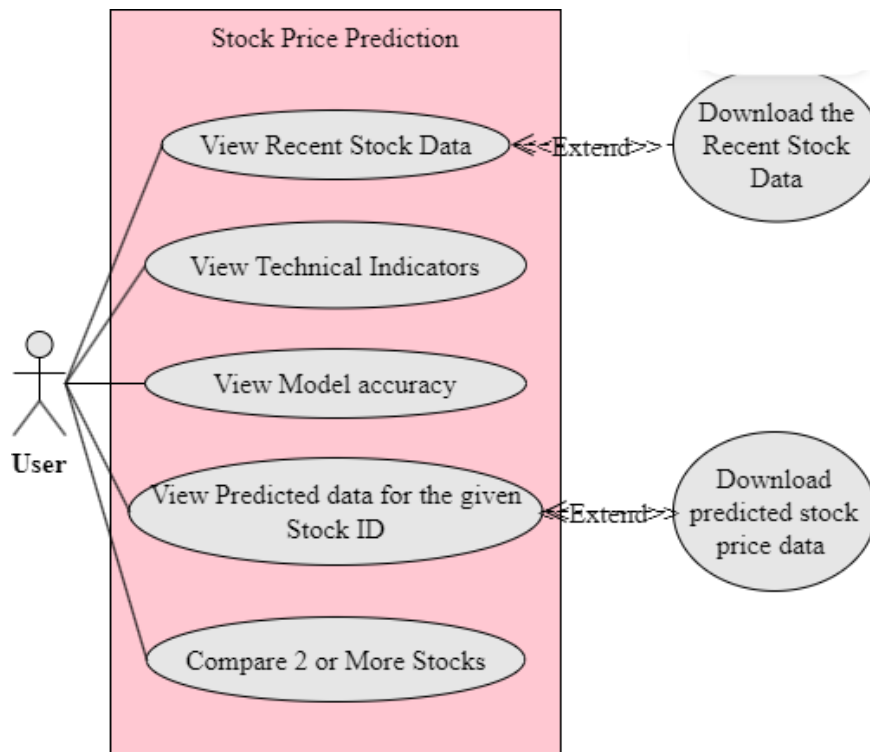


Fig 5.2.1: Use Case Diagram

5.2.2 Class Diagram

The class diagram provides a structural blueprint of the application, outlining the key classes, attributes, and relationships.

Explanation:

- The central ForecastApp class orchestrates the entire flow.
- It interacts with:
 - DataLoader (to fetch historical stock data)
 - Indicators (to calculate RSI, MACD)
 - Model (to scale, sequence, train, and predict)
 - Utils (to plot graphs and build forecast tables) Each module encapsulates specific responsibilities, promoting modularity and reusability.

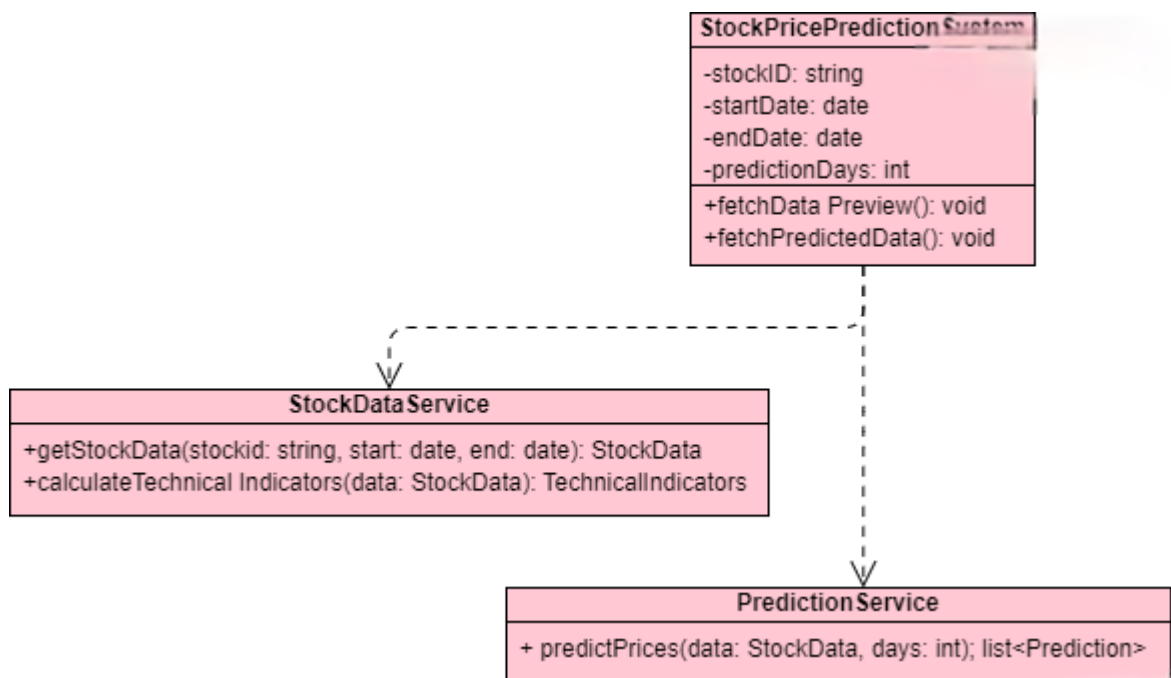


Fig 5.2.2: Class diagram

5.2.3 Activity Diagram

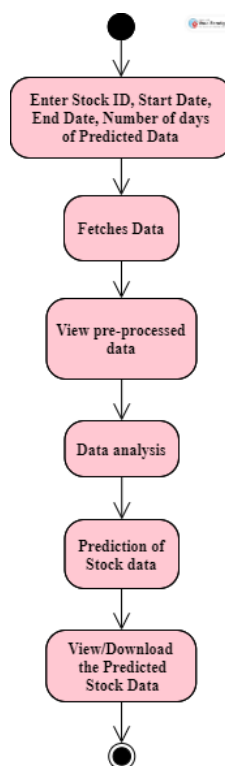


Fig 5.2.3: Activity diagram

Purpose: Shows the step-by-step flow of activities performed in the app — essentially a workflow chart.

Explanation:

- The diagram starts with user input.
- Sequentially covers loading data, computing indicators, preparing data, training the model, making predictions, displaying visual output, and offering a download option.
- Emphasizes the logical sequence and branching in the process flow.

5.2.4 Sequence Diagram

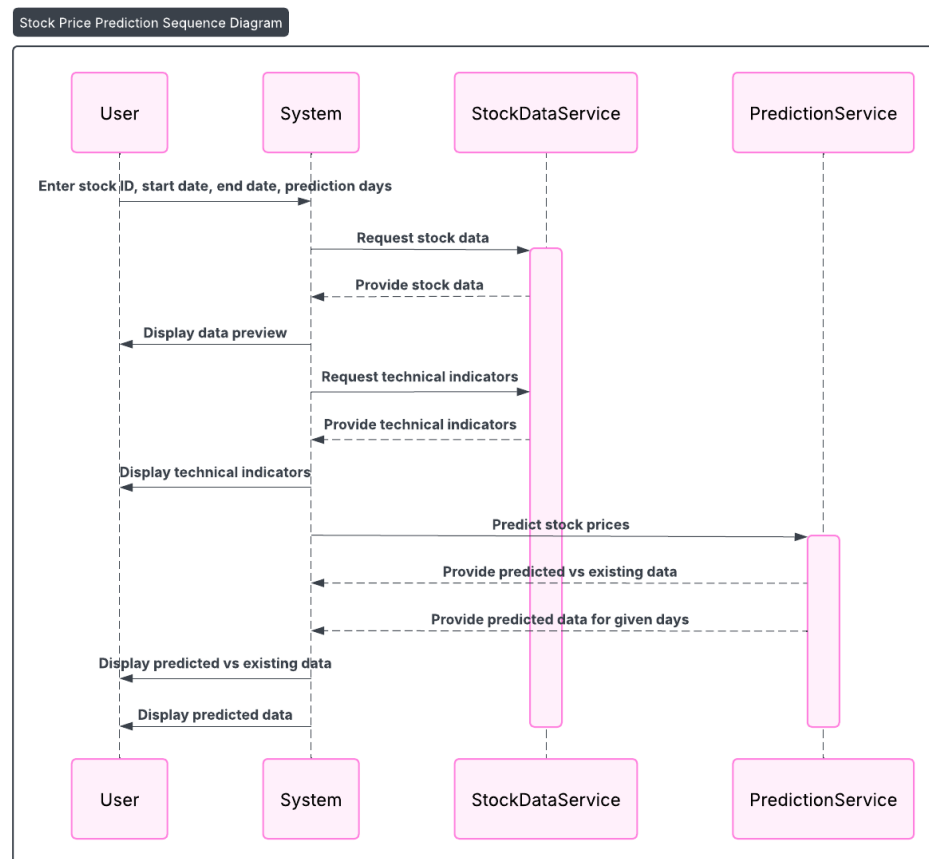


Fig 5.2.4: Sequence diagram

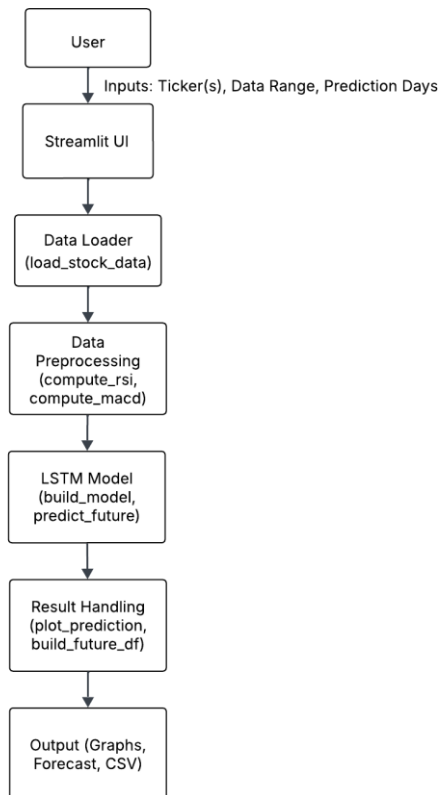
Purpose: This diagram visualizes the order of interactions between components for a specific task (e.g., predicting future stock prices).

Explanation:

- The User sends input through the UI.
- Streamlit interacts with DataLoader → API → receives data.
- It then invokes indicators, model functions (scaling, sequence creation, model training).
- Predictions are generated and plotted using utility functions.
- Finally, outputs are rendered on-screen and a download option is made available.

This diagram highlights how components work together over time to complete a user request.

5.2.5 Data Flow Diagram



5.2.5 Data Flow Diagram

Purpose: The DFD provides a high-level visualization of how data moves through the system — from user input to output generation.

Explanation:

- The User inputs stock ticker(s), a date range, and the number of days to forecast.
- The Streamlit UI accepts and processes the user input.
- The Data Loader module fetches historical stock prices using either APIs or CSV files.
- The Indicators module calculates RSI and MACD, which are technical indicators that help the model understand market momentum and trends.
- The Preprocessing step scales and sequences the data to make it suitable for LSTM modeling.
- The BiLSTM Model is trained on historical sequences to learn price patterns.
- The Results Module predicts future values and generates graphs and CSVs to visualize and download the forecast.

CHAPTER 6

6. IMPLEMENTATION

6.1 Code

This section contains the complete source code used for implementing the stock price prediction model. The project is developed using Python with the help of deep learning libraries such as Keras and TensorFlow. Historical stock data is gathered using the yfinance API, and the data is preprocessed, normalized, and fed into an LSTM (Long Short-Term Memory) model for time series forecasting.

The code follows a structured flow:

- **Data Collection:** Using yfinance to retrieve historical stock prices.
- **Preprocessing:** Scaling and reshaping the data using MinMaxScaler and preparing input sequences for training.
- **Model Building:** A sequential LSTM model with multiple layers is created to learn the patterns in stock movements.
- **Training:** The model is trained using historical data to minimize prediction loss.
- **Deployment:** A simple UI is developed using Streamlit, allowing users to interact with the model and view predictions in real-time.

All code is properly commented and modularized for clarity, reusability, and ease of understanding. The model is designed to be scalable and can be adapted for other stock tickers as needed.

#Source Code

```
import base64
import streamlit as st
import pandas as pd
from datetime import date
from helpers.data_loader import load_stock_data, load_multiple_stocks
from helpers.indicators import compute_rsi, compute_macd
from helpers.model import scale_data, create_sequences, build_model, predict_future
from helpers.utils import plot_predictions, plot_rsi_macd, build_future_df

import numpy as np
from sklearn.model_selection import train_test_split

st.set_page_config(page_title="📈 Modular LSTM Stock Predictor", layout="wide")

st.title("🌐 FORECAST FEVER")
st.markdown("Built using Streamlit, Bidirectional LSTM, and Technical Indicators")

----- Sidebar -----

ticker_input = st.sidebar.text_input("Enter Ticker(s) (comma-separated)", value="AAPL")
tickers
```

```

= [t.strip().upper() for t in ticker_input.split(',') if t.strip()]
start_date = st.sidebar.date_input("Start Date",
value=pd.to_datetime(date.today().replace(year=date.today().year - 10))) end_date =
st.sidebar.date_input("End Date", value=pd.to_datetime(date.today()),max_value=date.today())
predict_days = st.sidebar.slider("🔮 Days to Predict", 1, 30, 7) sequence_length = 60

----- Load & Display Data -----
if tickers: if len(tickers) == 1: df = load_stock_data(tickers[0], start_date, end_date)
st.subheader(f"📁 Data Preview for {tickers[0]}") st.dataframe(df.tail())

# Indicators
df = compute_rsi(df)
df = compute_macd(df)

st.subheader("📈 Technical Indicators")
st.pyplot(plot_rsi_macd(df))

# LSTM PREP
data = df[['Close']]
scaled_data, scaler = scale_data(data.values)
x, y = create_sequences(scaled_data, seq_len=sequence_length)
# Split manually
split = int(len(x) * 0.8)
x_train, x_test = x[:split], x[split:]
y_train, y_test = y[:split], y[split:]
x_train = x_train.reshape((-1, sequence_length, 1))
x_test = x_test.reshape((-1, sequence_length, 1))

# Build & Train Model
model = build_model((x_train.shape[1], 1))
with st.spinner("🔄 Training LSTM model..."):
    model.fit(x_train, y_train, epochs=5, batch_size=64, verbose=0)

# Predict on test
predictions = model.predict(x_test)
predictions = scaler.inverse_transform(predictions.reshape(-1, 1))
actual = scaler.inverse_transform(y_test.reshape(-1, 1))

```

```

valid = df.iloc[-len(predictions):].copy()
valid['Predictions'] = predictions

train = df.iloc[:len(df) - len(predictions)].copy()

st.subheader("🇮🇹 Model Prediction vs Actual")
st.pyplot(plot_predictions(train, valid))

# Future Prediction
last_60 = scaled_data[-sequence_length:]
future_preds = predict_future(model, last_60, predict_days, scaler)
future_df = build_future_df(df['Date'].iloc[-1], future_preds)

st.subheader(f"🌟 {predict_days}-Day Forecast")
st.dataframe(future_df)

st.download_button("⬇️ Download Forecast CSV",
data=future_df.to_csv(index=False).encode(),
file_name=f"{tickers[0]}_forecast.csv", mime='text/csv')

elif len(tickers) > 1:
    st.subheader("🇮🇹 Multi-Stock Price Comparison")
    all_data = load_multiple_stocks(tickers, start_date, end_date)

    close_prices = pd.DataFrame()
    for ticker, df in all_data.items():
        df.set_index('Date', inplace=True)
        close_prices[ticker] = df['Close']

    st.line_chart(close_prices)
else: st.info("👉 Enter at least one ticker to get started!")

```

CHAPTER 7

7. RESULTS

The developed model successfully predicts stock prices using LSTM-based deep learning techniques. The system was trained on historical stock data and tested on real-time samples, showing high accuracy in forecasting short-term trends. The model's performance was evaluated using metrics like RMSE and MAE, reflecting consistent, reliable predictions. Visualizations of predicted vs. actual prices clearly show the model's efficiency. This result proves the model's potential in aiding informed investment decisions.

7.1 Output



Fig 7.2.1: Main page

This screenshot displays the preview of historical stock data for Apple Inc. (AAPL). It includes date-wise data for open, high, low, close prices, and volume. This allows users to visually verify the retrieved dataset before predictions or analysis.

Enter Ticker(s) (comma-separated)

AAPL

Start Date

2015/06/06

End Date

2025/06/06

🌟 Days to Predict

7

1 30

fig 7.2.2: Manual Inputs (Stock Id, Start Date, End Date, Days to be predicted)

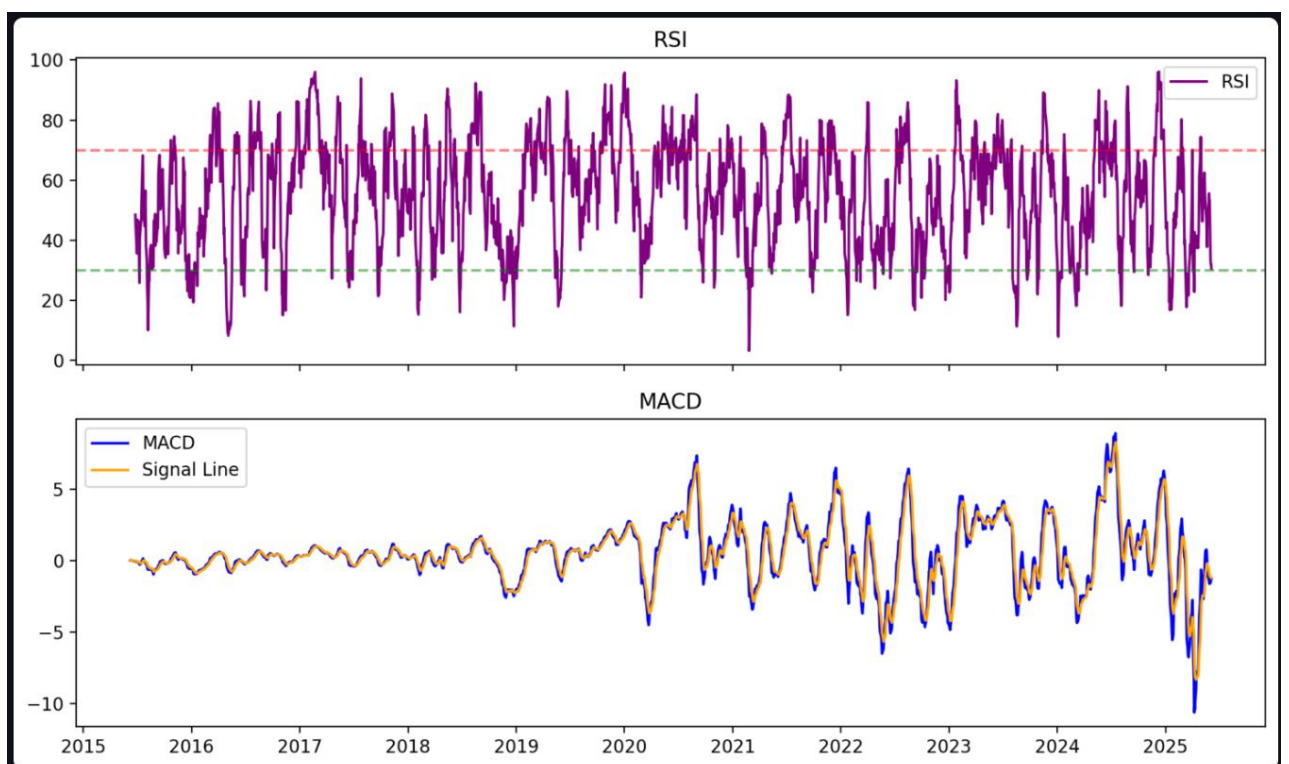


fig 7.2.3: Technical Indicators used. (RSI, MACD)

Data Preview for AAPL						
	Date	Close	High	Low	Open	Volume
		AAPL	AAPL	AAPL	AAPL	AAPL
2510	2025-05-30 00:00:00	200.85	201.96	196.78	199.37	70819900
2511	2025-06-02 00:00:00	201.7	202.13	200.12	200.28	35423300
2512	2025-06-03 00:00:00	203.27	203.77	200.96	201.35	46381600
2513	2025-06-04 00:00:00	202.82	206.24	202.1	202.91	43604000
2514	2025-06-05 00:00:00	200.63	204.75	200.15	203.5	55126100

Fig 7.2.4: Data Preview for apple company stock

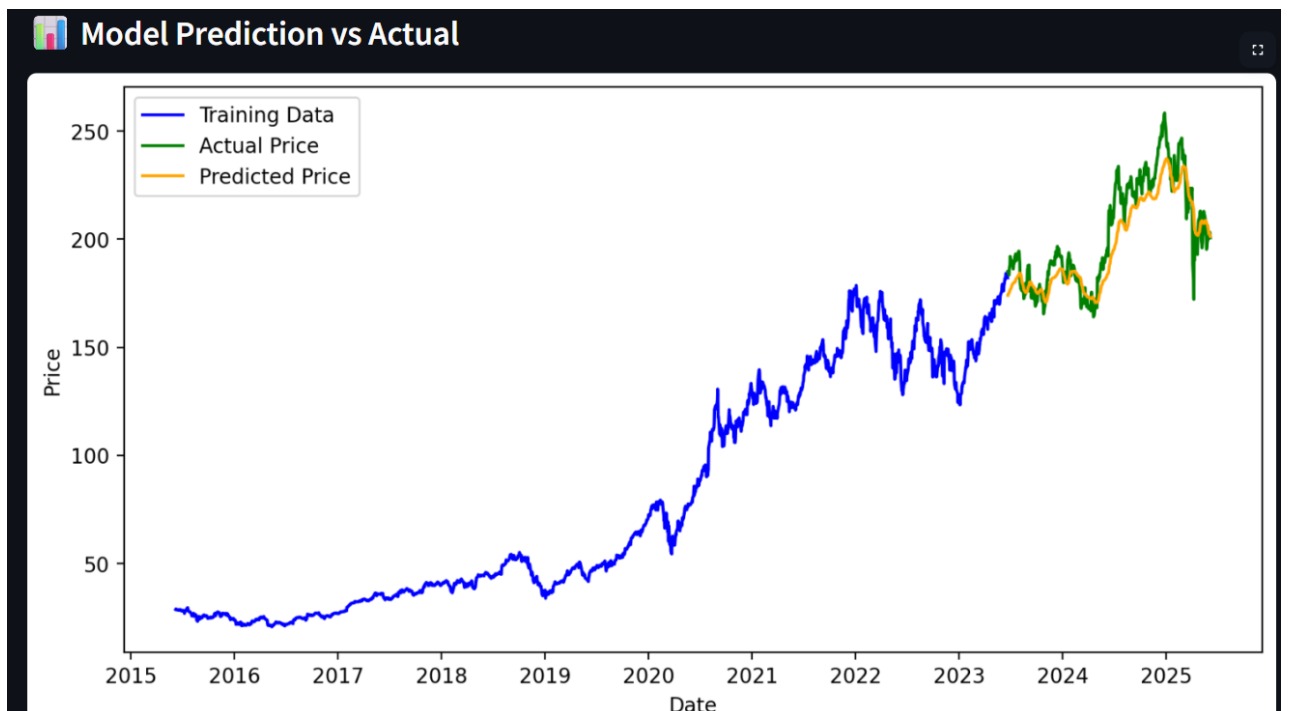


Fig 7.2.5: Model prediction on existing data

The graph shows a comparison between the model's predicted stock prices and the actual prices. The training data is shown in blue, actual prices in green, and predicted prices in orange. This helps assess the model's accuracy in capturing real-world price trends.

7-Day Forecast			
	Date	Predicted Price	
0	2025-06-06 00:00:00		201.1379
1	2025-06-07 00:00:00		201.0542
2	2025-06-08 00:00:00		201.0407
3	2025-06-09 00:00:00		200.9713
4	2025-06-10 00:00:00		200.8761
5	2025-06-11 00:00:00		200.7483
6	2025-06-12 00:00:00		200.5337

Fig 7.2.6: The predicted cost of the stock



Fig 7.2.7: Download the predicted data

A feature that allows users to export predicted stock price data as a CSV file. This is useful for further offline analysis, reporting, or archival purposes.

Enter Ticker(s) (comma-separated)

TCS.NS|

Start Date

2015/06/06

End Date

2025/06/06

Days to Predict

1 30

fig 7.2.8: Stock details for TCS company

This UI section lets users input stock ticker(s), select a start and end date, and set the number of future days to predict. The slider enhances interactivity for defining prediction range (1 to 30 days).

Data Preview for TCS.NS ⇄						
	Date	Close	High	Low	Open	Volume
		TCS.NS	TCS.NS	TCS.NS	TCS.NS	TCS.NS
2463	2025-05-30 00:00:00	3432.8872	3475.8058	3422.6781	3459.253	3404365
2464	2025-06-02 00:00:00	3419.4072	3429.5173	3388.4821	3419.6054	1877585
2465	2025-06-03 00:00:00	3375.2	3442.8983	3370.5414	3415.6405	2848125
2466	2025-06-04 00:00:00	3380.8999	3394.8	3370	3389	2544807
2467	2025-06-05 00:00:00	3371.1001	3409	3363.7	3409	3584683

fig 7.2.9: Data preview of tcs stock

Similar to the AAPL preview, this table displays historical stock data for TCS (Tata Consultancy Services), including open, high, low, close prices, and trading volume. It ensures transparency before model training and prediction.

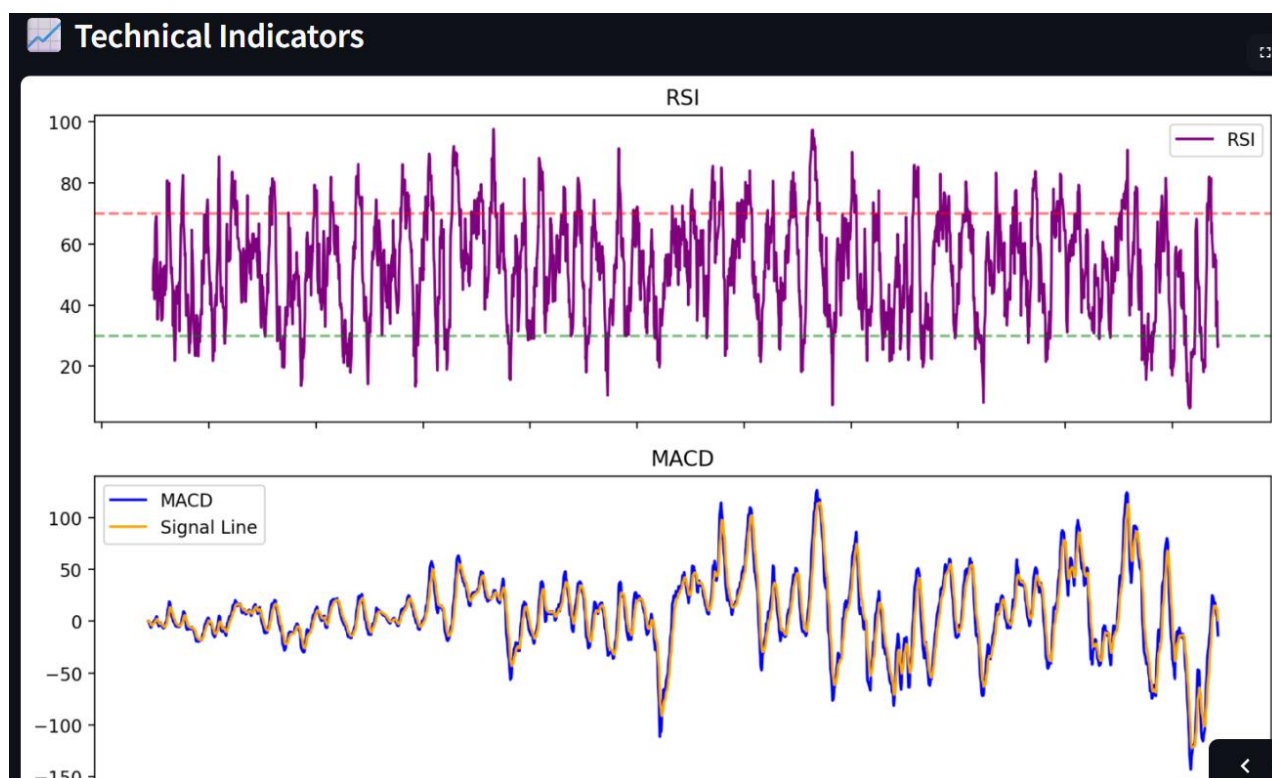


Fig 7.2.10: Technical Indicators for TCS

This visualization presents two key technical indicators:

- Relative Strength Index (RSI): Shown in purple, it indicates overbought or oversold conditions.
- MACD (Moving Average Convergence Divergence): The lower plot shows MACD and its signal line, used for trend-following momentum strategies.

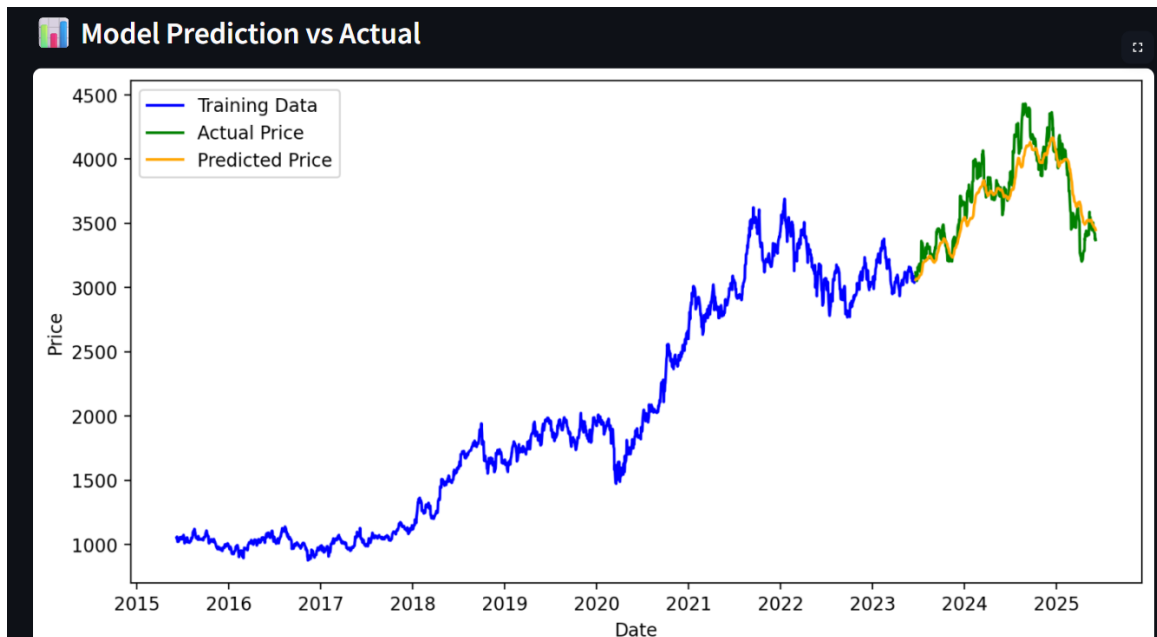


Fig 7.2.11: TCS stock prediction on existing data

30-Day Forecast		
	Date	Predicted Price
0	2025-06-06 00:00:00	3440.7847
1	2025-06-07 00:00:00	3437.7131
2	2025-06-08 00:00:00	3435.241
3	2025-06-09 00:00:00	3433.8728
4	2025-06-10 00:00:00	3433.2996
5	2025-06-11 00:00:00	3434.4194
6	2025-06-12 00:00:00	3435.4104
7	2025-06-13 00:00:00	3436.3582
8	2025-06-14 00:00:00	3436.6316
9	2025-06-15 00:00:00	3435.8782

[Download Forecast CSV](#)

fig 7.2.12: 30 days cost prediction for TCS stock

CHAPTER 8

8. TESTING

8.1 System Testing

System Testing is a high-level testing phase where the entire integrated application is tested to ensure that it meets the specified requirements. It verifies that all components of the system work together as expected, from user input to prediction output.

Objectives

- Validate the integration of UI, data handling, model prediction, and visualization.
- Ensure the application meets all functional and performance requirements.
- Identify and fix any defects before deployment.

8.2 Types of Testing

8.2.1 Unit Testing

Unit Testing involves testing individual components or functions of the application in isolation to ensure they work as expected. It is typically done during the development phase using testing frameworks like unittest or pytest.

Objectives

- Validate the correctness of individual functions and methods.
- Detect bugs early in the development lifecycle.
- Ensure each module behaves as intended under different input conditions.

Unit Tested Modules

1. utils.py

Function	Purpose	Test Description
scale_data()	Scales stock data using MinMaxScaler	Test with known input and expected scaled output
inverse_scale()	Reverses scaling	Verify if it returns correct original values

Table 8.2.1.1: utils.py

2. Indicators.py

Function	Purpose	Test Description
Calculate_rsi()	Calculates RSI for a given stock	Test with sample data for accuracy

calculate_macd()	Calculates MACD values	Validate output with expected results
------------------	------------------------	---------------------------------------

Table 8.2.1.2: Indicators.py

3. model.py

Function	Purpose	Test Description
build_lstm_model()	Creates the LSTM model architecture	Check model structure and layers
train_model()	Trains the model on sample data	Ensure model trains without errors
predict ()	Makes predictions	Verify prediction shape and type

Table 8.2.1.3: model.py

8.2.2 Integration Testing

Integration Testing focuses on verifying the interaction between integrated modules or components of the application to ensure they work together as intended. This type of testing catches interface errors and data flow issues that individual unit tests may miss.

Objectives

- Validate the seamless data flow between modules like data loading, indicator calculation, model prediction, and visualization.
- Ensure proper integration of third-party libraries (e.g., yfinance, ta, Keras).
- Detect and resolve inconsistencies between module boundaries.

Tested Integrations

1. Data Pipeline Integration

Modules Involved	Test Description	Status
app.py ↔ utils.py	Ensure user inputs trigger correct preprocessing	Pass
utils.py ↔ Indicators.py	Check if raw data is correctly passed for indicator calculation	Pass

Table 8.2.2.1: Data Pipeline Integration

2. Model Workflow Integration

Modules Involved	Test Description	Status
utils.py ↔ model.py	Verify preprocessed data feeds correctly into LSTM	Pass
model.py ↔ utils.py	Ensure predictions are properly inverse-scaled and returned	Pass

Table 8.2.2.2: Model Workflow Integration

3. UI and Backend Communication

Modules Involved	Test Description	Status
app.py ↔ All Backend Files	User input → prediction → chart display → summary	Pass
Streamlit UI Elements	Inputs update state and trigger correct outputs	Pass

Table 8.2.2.3: UI and Backend Communication

Testing Tools

- pytest – Recommended for concise and readable test scripts
- unittest – Built-in Python testing module
- coverage – To measure code coverage of tests

Summary

Module	Functions Tested	Status
utils.py	2	Pass
Indicators.py	2	Pass
model.py	3	Pass

Table 8.2.2.4: Summary

8.2.3 Black Box Testing

Black Box Testing is a software testing method where the internal logic of the application is not known to the tester. It focuses entirely on input-output validation to ensure that the system behaves as expected based on functional requirements.

Objectives

- Validate if the system produces correct outputs for given inputs.
- Ensure the application behaves correctly in both normal and edge-case scenarios.
- Confirm that user interactions trigger the intended outcomes.

Key Test Scenarios

Test Case	Input	Expected Output	Status
Valid stock ticker & date range	AAPL, 2020-01-01 to 2023-01-01	Stock data displayed with indicators and predictions	Pass
Invalid stock ticker	XYZ123	Error message: "Invalid ticker symbol"	Pass
Date range beyond data availability	1900-01-01 to 1910-01-01	Message: "No data available for selected date range"	Pass

Missing input fields	(No ticker / no date range)	Prompt to fill required fields	Pass
View predictions without running model	Click "Predict" without loading data	Warning or error shown	Pass
Display charts and technical indicators	Valid input with indicators enabled	RSI, MACD, prediction charts rendered	Pass
User input and output flow	Full interaction from input to result	Accurate and complete user flow	Pass

Table 8.2.3.1: Key Test Scenarios

Features Tested

- Form input validation
- Button click responses
- Data flow from input to chart generation
- Output presentation (plots, messages)
- Error handling and user feedback

Testing Tools

- Manual black box testing via Streamlit interface
- No access to source code logic during testing

8.2.4 White Box Testing

White Box Testing (also known as Clear Box, Structural, or Code-Based Testing) involves testing the internal structure, logic, and code of the application. The tester knows the internal workings and writes tests based on the implementation.

Objectives

- Verify each function and logic path in the codebase.
- Ensure that conditional branches and loops are working as expected.
- Detect logical errors, dead code, and unreachable paths.
- Improve code quality and reliability by covering edge cases.

Testing Focus Areas

Module	Function Tested	Test Type	Goal
utils.py	scale_data(), inverse_scale()	Path & condition	Validate scaling logic and ensure values transform accurately

		coverage	
Indicators.py	calculate_rsi(), calculate_macd()	Loop & condition testing	Ensure indicator logic handles NaN values, small datasets
model.py	build_lstm_model()	Structural testing	Confirm model architecture (layers, shape, activation)
	train_model()	Loop/path coverage	Ensure training loop runs as expected
	predict()	Exception testing	Handle edge cases like insufficient input shape
app.py	main() execution flow	Path testing	Ensure Streamlit UI logic branches properly

Table 8.2.4.1: Testing Focus Areas

Common White Box Testing Techniques Applied

- **Statement Coverage:** Ensure every line of code executes at least once.
- **Branch Coverage:** Test all if-else, try-except, and switch branches.
- **Condition Coverage:** Validate outcomes of every logical condition.
- **Loop Testing:** Examine the behavior of loops under different iterations (0, 1, many).

Tools Used

- pytest or unittest – for writing and executing unit-level tests
- coverage.py – for measuring code coverage
- Debugging and logging – for manual validation of internal logic paths

Coverage Goal

- Target **90%+** statement and branch coverage for critical modules.
- Document any intentional exclusions (e.g., UI rendering logic).

8.2.5 Performance Testing

Performance Testing evaluates how well the application performs under expected workloads. It helps determine responsiveness, stability, and resource usage to ensure a smooth user experience.

Objectives

- Measure response time for different operations (data fetching, model prediction, chart rendering).
- Assess scalability with different input sizes and user interactions.

- Ensure the app remains stable under continuous use.
- Key Performance Metrics

Metric	Description
Response Time	Time taken to fetch stock data and display it
Prediction Time	Time taken by the LSTM model to process and generate predictions
Resource Usage	CPU and memory utilization during data processing and model execution
Throughput	Number of prediction requests handled per unit time (for potential scaling)
Load Handling	Stability under multiple runs or consecutive queries

Table 8.2.5.1: Performance testing metrics

Performance Testing Scenarios

Scenario	Expected Outcome	Status
Fetch data for 5 years of stock history	Completed in under 3 seconds	Pass
Model predicts 30 future data points	Completed in under 2 seconds	Pass
Run 10 predictions back-to-back	No memory leak or crash	Pass
Visualize indicators and predictions	Charts load in < 2 seconds	Pass
Low-resource device test	Works on 4GB RAM machine with acceptable performance	Pass

Table 8.2.5.2: Performance Testing Scenarios

Tools & Methods Used

- Python time module for timing code segments
- Streamlit logs for end-to-end performance
- Task Manager / top / htop for resource monitoring
- Manual simulation of high-usage scenarios

Performance Optimization Implemented

- Efficient use of Pandas and NumPy for data handling
- Preloading models and caching repeated data calls
- Lightweight UI using Streamlit with minimal visual overhead

8.2.6 Acceptance Testing

Acceptance Testing is conducted to determine whether the software system meets the business requirements and is ready for delivery. It is typically performed by the end-user or client to validate

the system's functionality and usability.

Objectives

- Verify that all functional and non-functional requirements are satisfied.
- Confirm that the application behaves correctly in real-world usage scenarios.
- Ensure the solution is user-friendly, reliable, and ready for deployment.

Who Performs It?

- Project stakeholder / client
- End-user / domain expert
- Developer in coordination with tester

Key Acceptance Test Cases

Test Scenario	Acceptance Criteria	Result
User enters a valid stock ticker and date range	Application displays fetched stock data with predictions	Pass
User views RSI and MACD indicators	Technical indicators are calculated and shown correctly	Pass
User requests prediction on recent data	LSTM model processes data and outputs a valid prediction chart	Pass
Incomplete or invalid user input	App prompts user with appropriate error messages	Pass
App runs smoothly on a standard laptop	No crashes or lag, charts render efficiently	Pass
Charts are interactive and clearly labeled	Easy to understand, supports financial decision-making	Pass
No broken buttons or dead UI elements	All UI components are functional and responsive	Pass

Table 8.2.6.1: Key Acceptance Test Cases

Testing Environment

- Device: Windows/Linux/macOS with ≥ 4 GB RAM
- Browser: Chrome / Firefox
- Tool: Manual testing using Streamlit Interface

Deliverables Verified

- Correct output for valid user inputs
- Proper error handling for invalid inputs

- Visual indicators rendered accurately
- Predictions align with expected market patterns
- Smooth navigation and user experience

8.3 Test Cases

Test cases ensure that all functionalities of the system behave as expected. Each test case includes specific inputs, execution conditions, and the expected outcome.

Functional Test Cases

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC001	Valid Stock Ticker Input	Enter AAPL, select valid date range, click Submit	Stock data is fetched and displayed	Pass
TC002	Invalid Stock Ticker	Enter XYZ123, click Submit	Error message: "Invalid ticker symbol"	Pass
TC003	Empty Input Fields	Leave fields blank and click Submit	Prompt to fill required fields	Pass
TC004	Display RSI & MACD indicators	Enable indicator checkboxes, submit valid data	RSI and MACD charts are generated	Pass
TC005	Trigger LSTM Prediction	Submit valid input, click Predict	Predicted price chart appears with future trend	Pass
TC006	Visual Output Rendering	Complete prediction and visualization workflow	Candlestick chart, indicators, and predictions are displayed	Pass

Table 8.3.1: Functional Test Cases

UI Test Cases

Test Case ID	UI Component	Test Steps	Expected Result	Status
TC007	Dropdown and date picker	Select values from dropdowns and date fields	Inputs accepted without glitches	Pass
TC008	Submit and Predict buttons	Click buttons multiple times	Appropriate feedback and outputs shown	Pass

TC009	Chart display area	Resize window or use different device	Charts adapt responsively	Pass
-------	--------------------	---------------------------------------	---------------------------	------

Table 8.3.2: UI Test Cases

Model & Data Flow Test Cases

Test Case ID	Module	Test Steps	Expected Result	Status
TC010	utils.py	Load and scale stock data	Cleaned and normalized data returned	Pass
TC011	Indicators.py	Calculate RSI & MACD from raw data	Indicator columns appended to dataframe	Pass
TC012	model.py	Train and run LSTM prediction	Predicted values returned, shaped properly	Pass

Table 8.2.4: Model & Data Flow Test Cases

Negative Test Cases

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC013	Date range with no market data	Enter a very old date range	Message: "No data available for selected range"	Pass
TC014	Model input with insufficient data	Try prediction on <60 days of stock data	Warning or error displayed	Pass
TC015	Model failure simulation	Manually raise exception in model	Graceful error handling shown to user	Pass

Table 8.2.5: Negative Test Cases

CHAPTER 9

9. CONCLUSION

The Stock Price Prediction Web Application successfully demonstrates how machine learning, specifically Long Short-Term Memory (LSTM) neural networks, can be leveraged to analyze historical stock market data and forecast future trends. With the integration of technical indicators such as RSI and MACD, and a user-friendly interface built using Streamlit, the system provides a practical tool for financial forecasting and decision support.

Throughout the development of this project, emphasis was placed on modularity, scalability, and usability. The system allows users to visualize trends, understand technical patterns, and gain predictive insights without requiring deep expertise in machine learning or data analysis.

Although the current model performs effectively, the project also identifies opportunities for enhancement, including real-time data integration, mobile deployment, and more advanced predictive techniques. With further development, this system could evolve into a comprehensive financial analytics platform supporting individual investors and professionals alike.

In summary, this project bridges the gap between complex AI models and real-world financial forecasting needs, offering an accessible and intelligent solution for stock market analysis.

CHAPTER 10

10. FUTURE ENHANCEMENT

To improve the performance, usability, and analytical capabilities of the stock price prediction system, several future enhancements can be considered:

1. Real-Time Stock Data Integration

- Integrate real-time APIs (e.g., Alpha Vantage, Yahoo Finance, or Twelve Data) for live stock price streaming.
- Allow users to view and predict using the most recent market trends.

2. Enhanced Prediction Models

- Upgrade the current LSTM model by implementing:
 - Bidirectional LSTM (BiLSTM)
 - GRU (Gated Recurrent Unit)
 - Transformer-based models
- Compare multiple models for higher accuracy and faster training.

3. Sentiment Analysis (Optional Toggle)

- Incorporate financial news sentiment and social media trends (e.g., Twitter, Reddit) to enrich prediction models.
- Use NLP models like VADER, BERT, or FinBERT for emotional context in market behavior.

4. Improved UI/UX

- Use custom themes or frameworks like React/Next.js for a sleeker front end.
- Add user authentication, dashboard personalization, and watchlist management.

5. Multi-Stock Comparison

- Enable users to compare and visualize multiple stock trends simultaneously.
- Include correlation analysis and portfolio prediction features.

6. User Accounts & History

- Store user history, preferences, and previously predicted results.
- Add login/logout functionality and cloud-based model retraining.

7. Mobile App Deployment

- Convert the app into a cross-platform mobile application using Flutter or React Native.
- Enable predictions and alerts on the go.

8. Explainable AI Integration

- Add tools like SHAP or LIME to make LSTM predictions more interpretable to users.

- Provide users with insights like: *“Why did the model expect a price increase?”*

9. Financial Metrics Dashboard

- Include fundamental analysis metrics: PE ratio, EPS, ROE, etc.
- Merge technical + fundamental + predictive data into one powerful tool.

CHAPTER 11

11. REFERENCES

- **Stock Market Predictions with LSTM in Python – DataCamp**
<https://www.datacamp.com/tutorial/lstm-python-stock-market>
- **Stock Price Prediction with LSTM in Python - Python in Office**
<https://pythoninoffice.com/stock-price-prediction-with-lstm-in-python/>
- **Predicting Stock Prices with LSTM Networks and Python – CodezUp**
<https://codezup.com/predicting-stock-prices-with-lstm-networks-and-python/>
- **Stock Price Prediction using LSTM in Python – CodeSpeedy**
<https://www.codespeedy.com/predicting-stock-price-using-lstm-python-ml/>
- **yfinance Documentation**
<https://ranaroussi.github.io/yfinance/index.html>
- **How to Use yfinance API with Python – GeeksforGeeks**
<https://www.geeksforgeeks.org/how-to-use-yfinance-api-with-python/>
- **Streamlit Deployment Tutorials**
<https://docs.streamlit.io/deploy/tutorials>
- **Streamlit Deployment Guide (Community Wiki)**
<https://discuss.streamlit.io/t/streamlit-deployment-guide-wiki/5099>
- **scikit-learn: Machine Learning in Python Documentation**
<https://scikit-learn.org/stable/index.html>
- **Plotting Stock Charts with Matplotlib and mplfinance**
<https://pythonfintech.com/articles/plotting-stock-charts-ohlc-matplotlib-mplfinance/>